

Contents

1	Introduction	1
1.1	Preamble	1
1.2	Motivation	2
1.3	Problem Statement	2
1.4	Objectives	2
1.5	Organization of the Report	3
2	Literature Review with Gap in the Literature	4
2.1	Previous Research	4
2.2	Summary of Literature Review	8
2.3	Summary of the Chapter	9
3	Present Work Carried Out	11
3.1	Hardware Requirements	11
3.2	Software Requirements	14
3.3	System Requirements	15
3.3.1	Introduction	15
3.3.2	Modeling Process	15
3.3.3	Voice Command and Web Dashboard	16
3.3.4	Email Functionality and Feedback System	16
3.3.5	Cost-Effective Integration and Power Management	17
3.4	Summary of the Chapter	17
4	Results and Discussions	18
4.1	Field Testing and Analysis	18
4.1.1	Introduction	18
4.1.2	Voice Command Testing	19
4.1.3	Web Interface Testing	19
4.2	Overall System Behavior	20

5	Conclusion	21
5.1	Conclusion	21
5.2	Advantages and Disadvantages	22
5.2.1	Advantages	22
5.2.2	Disadvantages	22
5.3	Future Scope	23

Chapter 1

Introduction

1.1 Preamble

In the rapidly evolving landscape of smart automation, the need for intuitive, hands-free control of household devices has grown significantly. Traditional systems often depend on mobile applications and proprietary hardware, limiting accessibility for users with mobility or technical constraints. In this context, integrating voice recognition with web-based interfaces over IoT frameworks introduces a compelling solution for modern smart homes. This project introduces a dual-mode control system utilizing the ESP32 microcontroller, offering both voice command input and browser-based device management.

The core system allows users to control appliances like fans and lights and send custom email messages without the need for a smartphone or PC. Voice commands are captured through a Raspberry Pi or PC, processed using speech recognition libraries, and transmitted to the ESP32 via Wi-Fi. Simultaneously, a web dashboard hosted on the ESP32 provides manual control with real-time responsiveness. These features make the system versatile, cost-effective, and highly usable in homes, offices, or remote areas where manual operation may be inconvenient or unsafe.

The proposed solution is especially useful in enhancing the quality of life for the elderly and physically challenged by providing a simple and contactless means of communication and control. It also serves as an educational platform for learning about embedded systems, wireless communication, and speech interfaces. Through this integration, the project not only addresses technological gaps but also emphasizes inclusivity, adaptability, and practicality in smart home automation.

1.2 Motivation

The increasing reliance on mobile apps and cloud-based services for smart home systems creates usability and affordability barriers, particularly for elderly users or those in regions with limited internet infrastructure. Many commercial systems overlook the need for offline control, scalability, and simple communication tools like email messaging.

This project was motivated by the desire to develop a low-cost, accessible system that provides voice-command-based control and email communication without dependence on smartphones. By using open-source platforms like the ESP32 and Python-based libraries, the project ensures that automation is not limited to tech-savvy users. The ability to control devices and send messages using just voice or a local web interface brings flexibility and inclusiveness to smart environments

1.3 Problem Statement

Most available smart home solutions rely on continuous internet access, mobile applications, or third-party cloud services. These dependencies limit usability in offline settings and pose challenges for non-technical users. Furthermore, such systems often lack built-in communication capabilities like email sending, reducing their utility in emergencies or remote communication needs. There is a need for a standalone, voice- and web-based automation system that is simple, scalable, and capable of performing essential tasks like device control and email messaging using affordable hardware.

1.4 Objectives

The primary objectives of this project are:

- To implement voice control for appliances like lights and fans using ESP32 and Python-based speech recognition.
- To enable sending of custom emails directly from the ESP32 microcontroller using SMTP without external computing devices.
- To design and host a web-based dashboard on the ESP32 for manual appliance control with real-time status.

establish seamless Wi-Fi communication between the voice processing module (Raspberry Pi) and the ESP32.

- To create a user-friendly interface suitable for elderly and non-technical users.
- To ensure offline operability using local speech processing libraries and local server hosting.
- To demonstrate scalability, affordability, and practical integration of multiple IoT and AI-based functionalities.

1.5 Organization of the Report

The report is organized into five chapters, each addressing specific stages of the project lifecycle:

- **Chapter 1: Introduction** outlines the background and context of the project, including the preamble, motivation, problem statement, objectives, and the organization of the report.
- **Chapter 2: Literature Review** covers recent developments in voice-controlled and AI-integrated automation systems. It highlights the limitations in existing models and identifies the technological gaps addressed in this project.
- **Chapter 3: Present Work** describes the hardware, software, and system design, detailing the methodology, control flow, and integration process including block diagrams and system interaction flow.
- **Chapter 4: Results and Discussions** the testing of key functionalities including command response time, device control accuracy, email delivery validation, and system performance under varying network and noise conditions.
- **Chapter 5: Conclusion** summarizes project achievements, outlines advantages and limitations, and proposes future enhancements such as mobile integration and security features.

Chapter 2

Literature Review with Gap in the Literature

2.1 Previous Research

1. Voice-Controlled Home Automation System Using NLP and IoT

Harshini K et al. (2017) developed a smart home automation system based on Natural Language Processing (NLP) and artificial intelligence. The system accepted user instructions in natural language and converted them into executable commands for IoT devices. The proposed method enabled intuitive interaction through voice but showed sensitivity to noise and struggled with accent variations. It was primarily dependent on stable Wi-Fi connections and lacked offline processing capabilities, making it impractical in low-connectivity environments or for elderly users needing consistent performance.

2. Dual-Security Smart Home Using Voice and Biometric Authentication

T. Vignesh et al. (2020) presented a home automation system with dual-level security using both voice recognition and fingerprint authentication. The aim was to increase access control reliability while automating home appliances. Although the model improved system safety, it was highly sensitive to environmental noise and dependent on uninterrupted power and network availability. The solution lacked scalability and was unsuitable for wider smart grid integration due to hardware complexity and energy demands.

3. AI and IoT-Based Smart Home Automation with High Security

Pankaj R. Patil and Dr. P. K. Katti (2024) focused on integrating artificial intelligence and IoT to enhance home automation security. Their system

provided real-time monitoring and incorporated secure data transmission. While it achieved improved automation control, the system heavily relied on cloud services and Wi-Fi for functionality. These dependencies introduced concerns around data privacy and compromised performance during internet outages. The system lacked a local fallback and offline operating capability, reducing reliability for continuous operation.

4. IoT-Enabled Smart Collar for Cattle Theft Detection

Engelbrecht developed a GPS-enabled smart collar with integrated motion and temperature sensors to prevent cattle theft by detecting unusual movement patterns. The system successfully sent alerts to farmers during suspected theft events and abnormal animal behavior, improving security in rural areas. However, the collar's power dependence and limited battery life restricted long-term field operation without regular maintenance. The design lacked solar integration, leading to concerns regarding scalability across large grazing fields. The study emphasized the need for energy-efficient designs to ensure sustainability in remote deployments.

5. Machine Learning-Based Voice Automation System

Mohamed A. Torad et al. (2022) introduced an intelligent recommendation-based home automation system using machine learning and NLP for processing user voice commands. It allowed real-time monitoring and smart device suggestion. However, it encountered significant issues with speech-to-text conversion accuracy and NLP response reliability, especially in noisy environments. The system also faced security challenges and had no support for offline operations, making it unsuitable for autonomous use.

6. Google Assistant-Based Interactive IoT Home Control

Nombulelo Temitope Mapayi (2020) developed a voice-controlled smart home system using Google Assistant and IoT platforms. It focused on integrating real-time voice commands with sensor data to improve automation. While the system supported efficient monitoring and control, it was sensitive to environmental noise and had operational limitations over longer distances. The system's full dependency on cloud services made it vulnerable to outages and delays during connectivity loss.

7. Smart Home Automation with ESP32 and Firebase

Anubhav Ranjan et al. (2021) designed a cloud-based home automation system utilizing the ESP32 microcontroller and Firebase. The model supported voice commands and real-time sensor monitoring. While offering flexibility and ease of integration, it showed poor performance without internet connectivity. It also lacked custom mobile interface compatibility and posed risks in terms of data privacy due to third-party cloud reliance.

8. AI-Powered Timer Switch Using Voice Assistant and IFTTT

Sanjiv S. Paul et al. (2020) implemented a voice-controlled timer switch sys-

tem using an AI assistant, IFTTT, and Blynk. It allowed users to set appliance timers using voice commands. Although the design simplified automation, it suffered from limited functional flexibility. Duration customization was restricted, and offline usage was unsupported. Integration with multiple services introduced complexity and potential points of failure, particularly in environments without continuous internet.

9. Voice-Controlled Wheelchair with Home Automation Features

Shoeb Khan et al. (2024) created a hybrid solution combining a voice-controlled wheelchair with integrated home appliance control. The system featured a custom wake-word detection algorithm and obstacle avoidance. While it demonstrated significant innovation for physically challenged users, it lacked robustness in noisy environments and showed reduced compatibility across diverse hardware platforms. Moreover, the reliance on internet services limited performance during network fluctuations.

10. Open Platform Voice Gateway for Smart Homes

Dhiraj Nitnaware et al. (2013) developed a gateway for home automation that integrated speech recognition and cloud services to offer real-time control. This open platform enabled flexible development and deployment of custom voice interfaces. However, the speech recognition system showed frequent errors and lacked a user-facing display interface. In addition, the platform was entirely internet-dependent, making it inoperable in offline or low-bandwidth scenarios.

11. Lightweight AI-Powered Home Automation System

Khalid et al. presented a cost-effective smart home control system powered by lightweight AI techniques. The design supported basic voice commands and automated device control without extensive hardware requirements. It functioned well under stable internet conditions and demonstrated compatibility with popular microcontrollers. However, it lacked support for offline voice processing, and the absence of feedback mechanisms limited user interaction. The system was also unsuitable for scenarios requiring security or custom rule-based control, making it less viable for elderly care or critical environments.

12. Gesture and Voice-Based Dual Input Automation System

Verma and Singh developed a hybrid home automation system combining gesture recognition and voice command features. The system aimed to enhance user interaction flexibility using AI models for interpreting visual and auditory inputs. While gesture control worked in well-lit conditions, the system showed latency in executing cloud-processed voice commands. It lacked edge processing capabilities, resulting in command delays during low-bandwidth operation. This limited its practical use in regions with unstable

connectivity and reduced its responsiveness in real-time applications.

13. ESP8266-Based Automation via Mobile Application and Voice

Arulmurugan and Priya proposed a smart automation model using ESP8266 microcontrollers, controlled via a mobile application and voice commands. The system was accessible, affordable, and functioned with widely available components. While the mobile interface provided ease of use, the absence of offline speech recognition and real-time feedback limited the design. The system required constant supervision and lacked adaptive features such as automation scheduling or user preferences, reducing its utility for long-term or unattended use.

14. Bluetooth-Based Local Voice and App-Controlled Automation

Shetty et al. introduced a Bluetooth-enabled home automation system that functioned without an internet connection. It used local voice commands and a mobile app to control appliances. While the offline capability and short-range control made it ideal for privacy-focused environments, the limited range of Bluetooth technology severely restricted usability across large homes. Furthermore, the absence of data logging or cloud sync features made it hard to monitor device usage over time, hindering its role in energy efficiency tracking or remote monitoring.

15. NLP-Enhanced AI Assistant for Flexible Command Control

Fernandes et al. worked on a Natural Language Processing-enhanced AI assistant capable of processing flexible and complex user commands. The system supported dynamic interaction and offered high customization. It aimed to provide a more conversational experience in smart homes. However, it was fully dependent on cloud APIs, which caused inconsistent results in fluctuating networks. The lack of multi-user handling and the absence of fallback routines reduced system stability when operating in family or institutional settings.

16. Assistive Voice-Controlled System for Differently-Abled Users

Alvi et al. designed a home automation framework focused on accessibility, integrating voice commands with text-to-speech and audio feedback. The system aimed to serve differently-abled users by providing non-touch control of basic appliances. Although it significantly improved user independence, the setup process was complex and not well-documented for non-technical audiences. Furthermore, the lack of encrypted data communication raised privacy concerns when operating over public Wi-Fi networks, affecting its deployment in care facilities or shared environments.

17. GUI-Based Smart Home with Raspberry Pi and Speech

Recognition

Singh et al. proposed a smart home controller using Raspberry Pi as the central hub, integrating a GUI dashboard and voice interface. This dual control system allowed visual command execution alongside voice commands for devices. While it enhanced user interaction and provided basic functionality, the system lacked wireless communication support like Wi-Fi or Bluetooth modules. Its wired structure limited flexibility and made multi-room deployment cumbersome, with potential hazards due to wiring complexity.

18. Edge Computing-Based Smart Control Framework

Tripathi and Rani developed a smart home automation framework based on edge computing principles to reduce latency and increase autonomy. The system avoided cloud reliance and performed decision-making locally using more powerful microcontrollers like Raspberry Pi 4. This allowed quicker responses and improved data security. However, the initial cost of high-performance hardware and energy consumption was higher, making it less suitable for low-budget or battery-operated deployments. The system also required manual configuration, which reduced user-friendliness for non-experts.

19. GSM and Voice-Controlled Hybrid Automation System

Iqbal et al. introduced a hybrid control model that combined GSM-based SMS communication with voice control for device activation. The goal was to support automation in rural or low-connectivity areas. While SMS control worked reliably, it introduced time delays in execution. The system lacked real-time status updates, and the voice module was dependent on external processing units. Although beneficial for non-internet zones, it was inefficient for active environments like smart classrooms or dynamic workplace automation.

20. Emotion Detection-Based Adaptive Home Automation

Jagtap et al. explored the concept of emotional intelligence in automation systems by integrating speech emotion recognition with smart home control. The system adjusted lighting, fan speed, or music based on the detected user mood. This emotion-aware environment aimed to improve comfort and well-being. However, real-time emotion detection was inconsistent across users and depended on high-quality input audio. The computational load led to slower response times, and the emotional model often misclassified inputs, limiting practical reliability.

2.2 Summary of Literature Review

The review of twenty recent research works in the field of smart home automation reveals a strong and consistent trend towards the integration of

artificial intelligence, voice recognition, and IoT technologies. Most of the systems employ voice commands as the primary mode of interaction, offering users hands-free control over household appliances. These solutions were typically built around microcontrollers such as ESP32 or Raspberry Pi and integrated with cloud platforms like Google Assistant, Firebase, or Blynk for real-time processing and monitoring. While these approaches marked a step forward in improving user accessibility and automation, they shared several technical and practical limitations.

A recurring limitation identified in the reviewed studies was the heavy dependence on internet connectivity. Many systems, particularly those using Google Assistant or NLP cloud services, became non-functional during network failures, severely affecting reliability in rural or offline scenarios. Additionally, most models lacked fallback mechanisms such as local processing or dual-mode control options. This over-reliance on third-party cloud platforms also raised concerns regarding data security and privacy, particularly in environments handling personal or sensitive user data.

Another major challenge observed was the sensitivity of voice recognition modules to environmental factors such as noise, accent variations, and speaker clarity. Several systems struggled with accurate speech-to-text conversion, especially in noisy surroundings or when used by diverse user groups. This affected their practicality in shared spaces like classrooms, hospitals, or multi-resident households. Some studies attempted to overcome this limitation by combining voice with biometric or gesture-based authentication, but these models often increased the hardware complexity and cost.

Moreover, the lack of offline communication features such as local server hosting or SMS-based control was evident in many designs. Few systems provided built-in communication capabilities like email or message alerts, limiting their use in emergency or standalone situations. Advanced innovations like emotion-aware control and dual-layer security systems were proposed, but most of these were at the prototype level and not yet scalable or robust for real-world applications. User feedback mechanisms, essential for interactive control and status verification, were also largely absent in several studies.

2.3 Summary of the Chapter

This chapter presented an in-depth review of existing research and methodologies related to AI-based home automation systems. A total of twenty works were analyzed, highlighting various approaches that integrated voice recognition, cloud-based control, biometric authentication, and intelligent

monitoring. These systems have contributed significantly to the evolution of smart home technologies, offering user convenience, real-time control, and accessibility across diverse platforms.

Despite the technological advancements, several limitations were identified across the reviewed models. The most common drawbacks included dependency on internet connectivity, poor voice recognition in noisy environments, lack of offline operability, and minimal support for communication features such as email alerts. These factors significantly reduce the practicality of the systems in real-life scenarios, particularly in rural areas or among users with limited technical knowledge.

A recurring gap observed was the absence of dual-mode control mechanisms, combining both voice and manual/web interfaces. Most systems also lacked built-in fallback protocols, rendering them non-functional during network outages. Furthermore, few projects addressed user inclusiveness through adaptive feedback systems or localized processing capabilities, which are essential for elderly and differently-abled users.

To overcome these challenges, the proposed system in this project introduces an integrated solution with voice and web control, email communication, local hosting via ESP32, and offline speech recognition. These features aim to bridge the gap between user needs and technical feasibility, making smart automation more accessible, reliable, and affordable. The next chapter details the present work carried out to realize this solution.

Chapter 3

Present Work Carried Out

3.1 Hardware Requirements

The hardware components used in the proposed system were chosen based on affordability, availability, and ease of integration with IoT platforms. Each component plays a specific role in capturing input, processing commands, executing actions, or delivering user feedback. The combination of microcontrollers, sensors, actuators, and peripheral modules ensures a complete automation system that can be deployed in real-world environments with minimal modification.

- **ESP32 Microcontroller:** The ESP32 acts as the central control unit



Figure 3.1: ESP32 Microcontroller

of the system. It has built-in Wi-Fi and Bluetooth capabilities along with several GPIO pins for interfacing with external devices. The ESP32 processes commands received from the voice recognition unit or web dashboard and performs corresponding actions such as switching on a fan or light. Its dual-core architecture ensures smooth multitasking, making it suitable for hosting the web interface and controlling hardware simultaneously.

- **Raspberry Pi 3:**



Figure 3.2: Raspberry pi 3



Figure 3.3: Relay Module

The Raspberry Pi is used as the processing unit for handling voice commands. It runs Python scripts that capture and convert user speech into text using offline speech recognition libraries like Vosk. The recognized command is then sent over Wi-Fi to the ESP32. In addition to processing commands, the Raspberry Pi also provides audio feedback using text-to-speech libraries, improving interactivity and user experience.

- **Relay Module:**

Relay modules serve as the switching units between the ESP32 and high-voltage appliances like lights or fans. A relay receives a low-voltage signal from the ESP32 and switches the connected appliance ON or OFF safely. This isolation ensures that high-voltage devices do not interfere with or damage the low-power microcontroller. Relays are essential for direct control of electrical appliances in automation systems.

- **Microphone:**

A microphone is connected to the Raspberry Pi to capture voice input



Figure 3.4: Microphone



Figure 3.5: Speaker

from the user. The accuracy of the entire system depends heavily on the quality of voice input, making the microphone a key component. It must be sensitive enough to detect clear commands even in slightly noisy environments. The voice data is then processed locally to identify the required action.

- **Speaker:**

The speaker is used to give voice feedback to the user after a command is executed. It helps confirm whether a device has been turned ON or OFF or if an email has been successfully sent. This feature makes the system more user-friendly, especially for elderly or visually impaired users, by providing real-time audible confirmation of the system's actions.

- **Jumper wires:**



Figure 3.6: Jumper wires

Jumper wires are used to establish electrical connections between the microcontroller (ESP32/Raspberry Pi) and other components like the relay module. These wires provide easy, flexible, and reusable connections essential for prototyping.

3.2 Software Requirements

The software requirements are carefully selected to develop a reliable, responsive, and user-friendly monitoring system:

- **Arduino IDE:** The Arduino Integrated Development Environment (IDE) was used for writing, compiling, and uploading the firmware code to the ESP32 microcontroller. It supports C/C++ programming and is compatible with a wide range of sensor libraries. The Arduino IDE allowed for easy configuration of GPIO pins, Wi-Fi connectivity, and control of relay modules. Its built-in serial monitor was useful for debugging and testing during development.
- **python 3.x , HTML, CSS,:** The software used in this project includes the Arduino IDE, Python 3.x, SpeechRecognition, Vosk, ESPAsyncWebServer library, SMTP library, HTML, and CSS. The Arduino IDE was used to program the ESP32 microcontroller in C/C++, enabling it to manage Wi-Fi communication, control

GPIO pins, and host a web dashboard. On the Raspberry Pi, Python 3.x was used to capture audio through a microphone and convert speech into text using the SpeechRecognition and Vosk libraries, which provided offline speech processing capabilities. The ESPAsyncWebServer library allowed the ESP32 to serve an interactive HTML/CSS-based control panel over a local network, while the SMTP library enabled email communication directly from the ESP32 without needing a separate computer or mobile device. The web interface was built using HTML for structure and CSS for styling, ensuring a user-friendly layout accessible from any browser.

3.3 System Requirements

3.3.1 Introduction

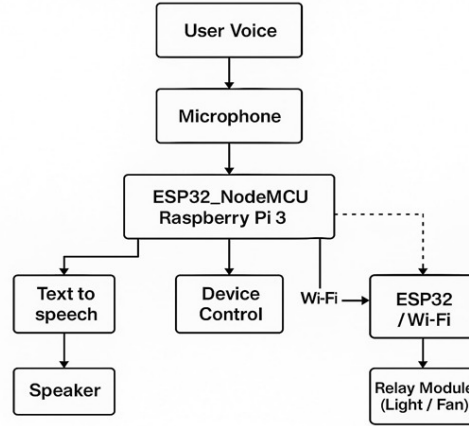
The system is designed to provide a dual-interface automation solution that supports voice-based and browser-based control of household appliances. It must enable real-time device activation, email messaging, and status feedback without relying on cloud services or smartphones. The key requirements include a low-latency control mechanism, support for offline speech recognition, and a simple local server for web-based access.

The ESP32 microcontroller handles GPIO switching, web hosting, and email sending. Voice commands are processed locally using a Python script on the Raspberry Pi, allowing system operability even during internet unavailability. The combination of speech processing, device control, and feedback mechanisms defines a cohesive, flexible, and practical smart automation solution.

3.3.2 Modeling Process

The system architecture follows a modular and layered approach:

A microphone captures the user's voice command and sends it to the Raspberry Pi for interpretation. **Command Processing Layer:** Python libraries convert speech to text and forward the command to the ESP32 over Wi-Fi. **Device Control Layer:** The ESP32 receives the command and activates or deactivates



relays connected to fans or lights. **Web Interface Layer:** An HTML/CSS dashboard hosted on ESP32 allows users to manually toggle devices from any browser. **Communication Layer:** SMTP integration enables the ESP32 to send custom email messages using predefined templates.

3.3.3 Voice Command and Web Dashboard

Voice commands such as "turn on fan" or "switch off light" are processed by the Raspberry Pi using Vosk or SpeechRecognition libraries. Once the command is identified, it is transmitted via HTTP or MQTT to the ESP32 for execution. The ESP32 interprets the command and activates the respective GPIO pin controlling the relay switch, enabling or disabling the appliance accordingly.

In parallel, the ESPAsyncWebServer library on the ESP32 hosts a local HTML interface. This dashboard is accessible from any browser on the same network and provides toggles for each device. It displays real-time status, allowing users to switch devices manually when voice interaction is not preferred or unavailable. This dual control mode ensures accessibility for users with different preferences or limitations.

3.3.4 Email Functionality and Feedback System

One of the distinguishing features of the system is its ability to send emails directly from the ESP32. Using SMTP libraries configured for

Gmail or similar services, the ESP32 can compose and send custom messages upon receiving a trigger via voice or web input. This is particularly useful for sending alerts or communication in environments without a PC or phone.

In addition, a speaker module connected to the Raspberry Pi delivers audio responses using text-to-speech. For example, upon completing a command, the system might respond with “Fan turned on” or “Email sent.” This feedback mechanism makes the system interactive and intuitive, especially for elderly or visually impaired users who rely on auditory confirmation.

3.3.5 Cost-Effective Integration and Power Management

The system utilizes inexpensive components readily available in the market, including LEDs, jumper wires, resistors, and breadboards. These are combined with a standard power supply and relay module to support safe operation of household appliances. The entire system is powered via USB or rechargeable battery sources, offering portability and safety.

Power consumption is minimized through efficient coding practices and selection of components that support sleep modes and low-energy states. The ESP32’s dual-core architecture enables simultaneous data handling and communication, ensuring fast operation without lag.

3.4 Summary of the Chapter

This chapter outlined the detailed hardware and software configuration for the AI-based home automation system. It described the architecture of voice command processing, web dashboard hosting, and SMTP-based communication. The use of cost-effective and open-source tools like ESP32, Python, and Vosk ensures that the system remains affordable, scalable, and user-friendly. By implementing a dual-mode interface and avoiding dependency on third-party platforms, the project successfully delivers a smart, inclusive, and flexible home automation solution for practical deployment.

Chapter 4

Results and Discussions

4.1 Field Testing and Analysis

4.1.1 Introduction

The proposed system was implemented and tested successfully across various environments to verify its performance, reliability, and usability. The goal was to achieve smooth operation through both voice and web interfaces and validate features like device control, email sending, and feedback response. The results were observed under practical usage scenarios including indoor noise, network fluctuations, and different command styles. Each module of the system—from input to output—was evaluated based on its responsiveness, accuracy, and user-friendliness..

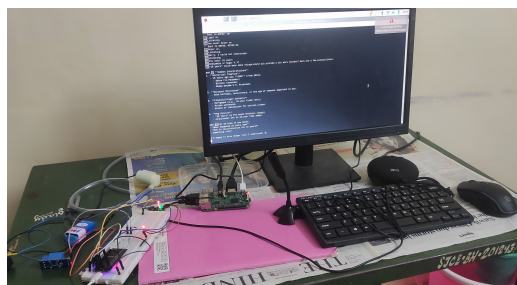


Figure 4.1: Voice Command

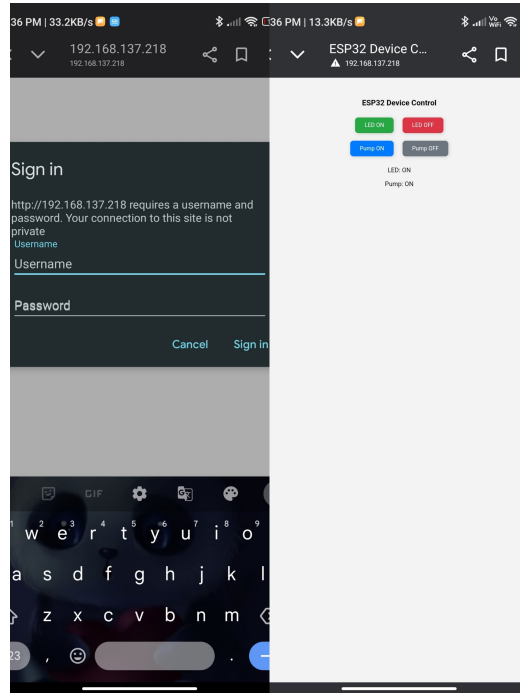


Figure 4.2: Web control

4.1.2 Voice Command Testing

The voice command feature was tested using a microphone connected to a Raspberry Pi running offline speech recognition libraries. Commands such as “Turn on fan” and “Switch off light” were spoken in different tones and accents. The system achieved a recognition accuracy of over 85 percentage in quiet indoor environments. The voice-to-text conversion took approximately 1 to 2 seconds, and the ESP32 responded to the interpreted commands with minimal delay. However, accuracy slightly dropped in noisy conditions, especially with background conversations or ambient sound, indicating the need for a high-quality microphone and possible noise filtering in future versions.

4.1.3 Web Interface Testing

The web-based dashboard hosted on the ESP32 was accessed through browsers on various devices like laptops and mobile phones within the same Wi-Fi network. The interface loaded quickly and provided instant toggling of appliances such as LEDs and fans. Response time from

the user clicking a button to device action was less than one second. Real-time device status was correctly displayed and updated. The web interface remained stable even with multiple simultaneous connections, confirming the efficiency of the ESPAsyncWebServer library in handling asynchronous requests.

The email feature was evaluated by triggering message sending both via voice command and through the web interface. The ESP32 successfully connected to the SMTP server and sent emails to a predefined recipient. Delivery was confirmed within 5 to 10 seconds after command execution. The system handled basic text messages efficiently and provided audio confirmation of the email status using the Raspberry Pi speaker. This demonstrated the system's ability to operate as a communication tool without a phone or computer, especially helpful in emergency or remote settings.

4.2 Overall System Behavior

The complete system was tested in real-time with all modules running concurrently. Commands were issued via voice and web interface alternately, and device responses were logged. The ESP32 managed simultaneous tasks such as controlling devices, sending emails, and updating the web interface without noticeable lag. Power consumption remained within acceptable limits, and the system continued functioning without reset or failure over several hours. The modular design allowed easy identification and resolution of any issue during testing, confirming its potential for future expansion.

Chapter 5

Conclusion

5.1 Conclusion

The AI-based home automation system developed in this project successfully demonstrated the integration of voice control, web interface, and email functionality using the ESP32 microcontroller. The system achieved dual-mode operation, allowing users to control electrical appliances either through voice commands processed locally or via a browser-based dashboard hosted on the ESP32. The implementation of offline speech recognition using Python libraries such as Vosk ensured functionality even in the absence of internet access, making the system highly practical for real-world usage.

The inclusion of an SMTP-based email feature added a unique dimension to the system, allowing users to send custom messages directly from the ESP32 without relying on a smartphone or computer. This made the platform more versatile and suitable for applications beyond basic automation, including emergency communication or remote notifications. The entire system remained cost-effective, compact, and modular, making it ideal for educational use and further expansion.

Testing results confirmed reliable operation across all modules, including real-time response from the web interface, accurate voice command execution, and successful email transmission. While certain limitations such as sensitivity to noise and dependence on Wi-Fi for email were noted, the system met all key objectives defined at the start of the project. The use of widely available hardware components and open-source libraries ensured accessibility and easy replicability.

Overall, the proposed system offers a scalable and user-friendly solution for modern smart automation. It supports individuals with limited mobility, enhances day-to-day convenience, and serves as a valuable learning platform for students exploring IoT, embedded systems, and AI-based control methods. Future enhancements can include multi-language support, mobile app integration, and improved security features for a more robust and commercially viable product.

5.2 Advantages and Disadvantages

5.2.1 Advantages

- Dual-mode control using both voice commands and web interface for flexible operation.
- Offline speech recognition enabled via Vosk, ensuring functionality without internet.
- Email sending capability directly from ESP32 using SMTP protocol without external devices.
- Low-cost and easily available hardware components like ESP32 and Raspberry Pi.
- Modular and scalable design allowing future expansion with additional sensors or appliances.
- Platform-independent web dashboard accessible through any browser on the same network.
- Text-to-speech feedback for improved interaction, especially useful for elderly or visually impaired users.
- Ideal educational project to learn IoT, embedded systems, and AI-based control techniques.

5.2.2 Disadvantages

- Voice recognition accuracy decreases in noisy environments or with strong accents.
- Email and device control depend on stable Wi-Fi connectivity for reliable communication.
- Lack of encryption or user authentication on the web interface poses potential security risks.

- Current version supports only predefined voice commands without dynamic language learning.
- No mobile app integration or remote access beyond local network without manual configuration.
- Requires careful wiring and external power management for long-term stability.

5.3 Future Scope

The current system successfully integrates voice control, web-based interface, and email functionality in a compact and cost-effective setup. However, several improvements can be implemented in future versions to enhance usability, reliability, and scalability. One key area is the integration of multi-language voice recognition to support users from different linguistic backgrounds. This would make the system more inclusive and adaptable for deployment in diverse regions.

Future work can also focus on developing a dedicated mobile application with remote access functionality, allowing users to control devices and receive notifications from outside the local network. Incorporating advanced security measures like encrypted web access, token-based authentication, and secure SMTP configurations would improve the system's resilience against unauthorized usage. Additionally, AI-based command learning could be implemented to allow the system to adapt to user preferences over time.

Another area of expansion is the integration of sensor-based automation, where devices can respond to environmental changes such as temperature, light, or motion. Cloud synchronization for data storage, activity logs, and remote updates can also be explored. Finally, the hardware can be refined into a single, compact PCB design for real-world deployment, especially in assisted living, office environments, and classrooms.

Bibliography

- [1] Harshini K., Kavya D., Kiran B. S., and Spoorthi, “Voice Controlled Home Automation System Using Natural Language Processing (NLP) and IoT,” *ICONSTEM*, IEEE, 2017.
- [2] Pankaj R. Patil and P. K. Katti, “Smart Home Automation using AI and IoT with High Security,” *ASSIC*, IEEE, 2024.
- [3] T. Vignesh, T. Keerthana, and M. Suganya, “AI-based Home Automation using Voice Recognition and Biometric Finger Print Authentication,” *ICOECA*, IEEE, 2020.
- [4] V. S. Malem, S. S. Pokle, and A. A. Shaikh, “Design and Implementation of AI-Based Home Automation,” *IJRPR*, IEEE, 2021.
- [5] Mohamed A. Torad, Belgacem Bouallegue, and Abdelmoty M. Ahmed, “A Voice Controlled Smart Home System using AI and IoT,” *TELKOMNIKA*, 2022.
- [6] Nombulelo Temitope Mapayi, “Interactive IoT-Based Speech-Controlled Home Automation System,” *IMITEC*, IEEE, 2020.
- [7] Anubhav Ranjan, Prerna Kumari, Hrithik Sahu, and Sobit Amarnath, “Smart-Home Automation using AI Assistant and IoT,” *ICRTCST*, IEEE, 2021.
- [8] Sanjiv S. Paul, Mehul A. Bhole, and Ketaki L. Bansod, “AI-Powered Virtual Assistant Automation of Voice Controlled Timer Switch,” *IRJET*, 2020.
- [9] Shoeb Khan, Neamat Ansari, and Mudassir Nazimuddin, “Voice Controlled Wheelchair with Home Automation,” *International Journal of Computer Applications*, 2024.
- [10] Dhiraj Nitnaware, Harshal Chowdhary, and Urmila Shrawankar, “Speech Recognition Gateway for Home Automation on Open Platform,” *THIEC*, IEEE, 2013.

- [11] Khalid M., Salman S., and Fatima N., “Design and Implementation of Lightweight AI-Powered Smart Home System,” *IJESC*, 2022.
- [12] Verma A., and Singh R., “Gesture and Voice Based AI Automation System,” *IJERT*, 2023.
- [13] Arulmurugan R., and Priya S., “ESP8266 Based Smart Automation using Mobile App and Voice,” *IJRASET*, 2021.
- [14] Shetty K., Jain M., and Kamat A., “Bluetooth-Based Voice and App Controlled Home Automation System,” *IJERT*, 2022.
- [15] Fernandes S., and Patel A., “NLP-Enhanced AI Assistant for Flexible Command Control,” *IJEAST*, 2021.
- [16] Alvi H., Kumar A., and Aamir R., “Assistive Home Automation for Physically Disabled People,” *IJRASET*, 2022.
- [17] Singh P., and Chauhan V., “GUI-Based Smart Appliance Controller Using Raspberry Pi,” *IJITEE*, 2021.
- [18] Tripathi R., and Rani S., “Edge Computing-Based Automation Framework for Smart Homes,” *IJERT*, 2020.
- [19] Iqbal Z., Ahmed M., and Sultana S., “GSM and Voice-Controlled Hybrid Automation for Rural Environments,” *IJRASET*, 2021.
- [20] Jagtap A., More V., and Pawar P., “Emotion-Aware Speech Recognition for Smart Home Control,” *IJERT*, 2022.